

HYMOEX: A Hybrid Modular Cognitive Architecture for Scalable Multi-Agent Expert Coordination

Joel Timana
Principal Researcher
joel@pymut.ai

Diana Munoz
Data Curation, Project Administration
diana@pymut.ai

Pymut Lab

January 20, 2026

Abstract

The proliferation of large language model (LLM)-based multi-agent systems has exposed fundamental limitations in existing orchestration frameworks: teams face architecture decision paralysis when selecting coordination patterns, expert coordination breakdowns cause redundant computation and misaligned outputs, systems exhibit brittle scalability that necessitates full rewrites upon growth, and framework fragmentation prevents knowledge transfer across implementations. We present HYMOEX (Hybrid Modular Coordinated Experts), a cognitive architecture for multi-agent systems that addresses these challenges through four contributions. First, we introduce a formal seven-role agent taxonomy—Manager, Integrator, Expert Manager, Supervisor, Expert, Executor, and Perceiver—with typed message protocols grounded in Clean Architecture principles. Second, we define three deployment modalities—One-Line MoE ($k \leq 2$ experts), One-Line Supervisor ($3 \leq k \leq 5$), and MoE MultiLine ($k > 5$)—that enable progressive scaling through formally specified migration functions preserving 100% of existing agent definitions. Third, we adapt the Mixture-of-Experts gating mechanism from neural networks to discrete agent coordination, achieving 96.7% expert selection accuracy versus 93.3% for rule-based routing in empirical benchmarks using Gemini 3 Flash. Fourth, we validate HYMOEX through controlled benchmarks and three production case studies spanning HR technology, real estate, and global workforce management, demonstrating 45% reduction in coordination token overhead and 93% token reduction at ten agents compared to flat multi-agent architectures. HYMOEX is framework-agnostic, with bidirectional adapters for seven major orchestration frameworks including LangGraph, CrewAI, and AutoGen.

1 Introduction

Multi-agent systems built on large language models (LLMs) have emerged as a dominant paradigm for solving complex, multi-step tasks that exceed the capabilities of individual models [7, 26]. From software development [21] to scientific research [19] and enterprise automation, the promise of specialized agents collaborating toward shared objectives has driven rapid adoption across industry and academia. Yet despite this enthusiasm, empirical evidence suggests that a substantial proportion of multi-agent deployments fail to reach production or degrade significantly upon scaling [25].

We identify four root causes behind these failures:

Architecture Decision Paralysis. Teams routinely spend weeks deliberating fundamental design questions: How many agents are needed? What coordination pattern—swarm, hierarchical, or hybrid—fits the task? Existing frameworks offer no principled decision framework, forcing ad-hoc choices that are expensive to revise. Recent taxonomies of multi-agent system failures identify architectural design problems as the largest category of deployment failures [7].

Expert Coordination Breakdown. When multiple specialized agents operate without structured coordination, misalignment manifests as redundant API calls, contradictory outputs, and wasted computation. Studies of LLM-based multi-agent conversations report that coordination failures account for over one-third of observable system errors [28], with token consumption scaling super-linearly in the number of uncoordinated agents.

Brittle Scalability. Systems that perform well with two or three agents frequently collapse when scaled to ten or more. Flat architectures suffer from quadratic communication complexity, while rigid hierarchies cannot accommodate new domains without structural rearchitecting. The result is a recurring pattern: successful prototypes that require complete rewrites for production scale.

Framework Fragmentation. The multi-agent ecosystem comprises over ten major frameworks—AutoGen [28], MetaGPT [8], CrewAI [2], LangGraph [12], OpenAI Swarm [18], among others—each imposing distinct coordination paradigms and abstraction levels. Organizations that adopt one framework face significant lock-in; knowledge gained in one system rarely transfers to another.

1.1 Contributions

We present HYMOEX (Hybrid Modular Coordinated Experts), a cognitive architecture that addresses all four challenges simultaneously. Our contributions are:

- C1. A formal seven-role agent taxonomy** with typed message protocols. We define agents as tuples with explicit roles, input/output spaces, and policies, grounded in Clean Architecture [17] and SOLID principles. This provides a universal vocabulary for describing multi-agent systems independent of implementation framework (Section 4.1).
- C2. Three graduated deployment modalities** with proven migration paths. We formalize One-Line MoE, One-Line Supervisor, and MoE MultiLine as increasingly expressive graph constraints, proving that each modality subsumes the previous (Section 4.2). The migration protocol preserves 100% of existing agent definitions, requiring only additive changes when scaling (Section 4.4).

C3. A Mixture-of-Experts gating mechanism adapted for discrete agent coordination.

We bridge the gap between neural MoE [4, 23] and multi-agent expert selection by defining a signal-vector-based gating function over agent pools, achieving 96.7% expert selection accuracy with a Rational-Raffle fusion strategy (Section 4.3).

C4. Empirical validation through benchmarks and production case studies. We report controlled benchmarks using Gemini 3 Flash demonstrating 45% reduction in coordination tokens and 93% token reduction at ten agents, alongside three production case studies spanning HR technology, real estate, and global workforce management (Section 6).

HYMOEX is framework-agnostic by design: we provide bidirectional adapters for seven major frameworks, enabling incremental adoption without lock-in. The architecture, reference implementation, and evaluation suite are publicly available.¹

1.2 Paper Organization

Section 2 surveys related work across multi-agent systems, cognitive architectures, and Mixture of Experts. Section 3 introduces formal preliminaries and notation. Section 4 presents the HYMOEX architecture in detail—our primary contribution. Section 5 describes the reference implementation and framework adapters. Section 6 reports experimental results, production case studies, and ablation studies. Section 7 discusses limitations and future work. Section 8 concludes.

2 Related Work

We situate HYMOEX at the intersection of three research threads: classical multi-agent systems, LLM-based agent frameworks, and Mixture of Experts architectures.

2.1 Classical Multi-Agent Systems

The foundations of multi-agent coordination were established through decades of work on agent architectures and interaction protocols [5, 27]. The BDI (Belief-Desire-Intention) model [22] formalized agent reasoning as a cycle of perception, deliberation, and action—a structure echoed in HYMOEX’s separation of Perceivers (sensing agents), Experts (domain specialists), and Executors (action agents), formally defined in Section 4.1. Cognitive architectures such as Soar [11] and ACT-R [1] demonstrated that modular, hierarchical processing enables both reactive and deliberative behavior within a unified framework.

HYMOEX draws from these traditions but targets a fundamentally different substrate: LLM-based agents that communicate through natural language rather than formal protocols, operate with stochastic rather than deterministic policies, and require explicit management of token budgets and API costs—concerns absent from classical architectures.

¹Repository: <https://github.com/Pymut/hymoex>

2.2 LLM-based Multi-Agent Frameworks

The rapid growth of LLM-based multi-agent systems has produced a diverse ecosystem of frameworks, each embodying distinct coordination paradigms:

Conversation-centric. AutoGen [28] models multi-agent interaction as structured conversations between agents, supporting both autonomous and human-in-the-loop modes. While flexible, the flat conversational topology lacks built-in mechanisms for progressive scaling or expert specialization.

Role-based. MetaGPT [8] assigns software engineering roles (product manager, architect, engineer) to agents following Standard Operating Procedures (SOPs). CAMEL [14] uses inception prompting to assign roles in communicative agent pairs. Both approaches fix the organizational structure at design time, making adaptation to different coordination patterns difficult.

Task-decomposition. HuggingGPT [24] and ChatDev [21] decompose complex tasks into sub-tasks dispatched to specialized models. These systems excel at pipeline-style workflows but do not support the dynamic expert selection central to HYMOEX.

Graph-based. LangGraph [12] represents agent workflows as state machines with conditional edges, providing fine-grained control over execution flow. However, LangGraph operates at the implementation level—it is a tool for building agent systems, not an architectural framework for classifying and migrating between coordination patterns.

Lightweight handoff. OpenAI Swarm [18] provides minimal abstractions for agent-to-agent handoffs, emphasizing simplicity over structure. While suitable for prototyping, it offers no mechanisms for hierarchical coordination, expert selection, or scalable deployment.

Table 1 summarizes the key differences. Critically, no existing framework provides: (a) a principled decision framework for choosing coordination patterns, (b) progressive migration between modalities without full rewrites, or (c) MoE-native gating for expert selection. HYMOEX addresses all three.

2.3 Mixture of Experts

The Mixture of Experts (MoE) paradigm, introduced by Jacobs et al. [9] and scaled to modern deep learning by Shazeer et al. [23], routes each input to a sparse subset of specialized sub-networks via a learned gating function. Switch Transformers [4] simplified the gating to top-1 selection, while GShard [13] demonstrated MoE scaling across distributed hardware.

HYMOEX adapts the MoE paradigm from continuous neural representations to discrete agent coordination. The key differences are: (i) “experts” are autonomous LLM agents with distinct system prompts and tool access rather than parameter blocks; (ii) the gating function operates over a composite signal vector (intent, emotion, stage, score) rather than

Table 1: Comparison of multi-agent frameworks across architectural dimensions. HYMOEX is the only framework providing all five capabilities simultaneously.

Framework	Progressive Scaling	MoE Gating	Agent Taxonomy	Framework Agnostic	Formal Protocol
AutoGen [28]	×	×	×	×	×
MetaGPT [8]	×	×	~	×	~
CrewAI [2]	×	×	~	×	×
LangGraph [12]	×	×	×	×	~
Swarm [18]	×	×	×	×	×
HYMOEX (ours)	✓	✓	✓	✓	✓

hidden-state embeddings; and (iii) fusion of expert outputs requires semantic merging of natural-language responses rather than weighted vector summation. We formalize these distinctions in Section 3 and describe our adapted gating mechanism in Section 4.

2.4 Software Architecture for Agent Systems

The application of software engineering principles to agent systems remains underexplored. Jennings [10] argued for agent-oriented software engineering as a paradigm, but concrete architectural patterns for LLM-based agents have lagged behind framework proliferation. HYMOEX explicitly adopts Clean Architecture [17] and SOLID design principles [16], enforcing single-responsibility through its role taxonomy and dependency inversion through typed message protocols. The Adapter pattern [6] underpins our framework-agnostic design, enabling bidirectional translation between HYMOEX abstractions and framework-specific APIs.

3 Background

We introduce the formal foundations upon which the HYMOEX architecture is built. We define agents, multi-agent systems, signal vectors, and gating functions, and establish the connection between neural Mixture of Experts and discrete agent coordination.

3.1 Agent Model

Definition 1 (Agent). *An agent is a tuple $a = (\text{id}, \rho, \mathcal{I}, \mathcal{O}, \pi)$ where:*

- $\text{id} \in \mathbb{N}$ is a unique identifier,
- $\rho \in \mathcal{R} = \{\text{MGR}, \text{INT}, \text{EXM}, \text{SUP}, \text{EXP}, \text{EXE}, \text{PCV}\}$ is the agent’s role (Manager, Integrator, Expert Manager, Supervisor, Expert, Executor, Perceiver),
- $\mathcal{I}$ is the input message space,
- $\mathcal{O}$ is the output message space,
- $\pi : \mathcal{I} \rightarrow \mathcal{O}$ is the agent’s policy function, which may be deterministic (rule-based) or stochastic (LLM-based).

Each role ρ imposes constraints on the allowed input and output types, enforcing the single-responsibility principle at the architectural level. We formalize these constraints in Section 4.

3.2 Multi-Agent System

Definition 2 (Multi-Agent System). *A multi-agent system (MAS) is a directed graph $\mathcal{G} = (\mathcal{A}, \mathcal{E}, \tau)$ where:*

- $\mathcal{A} = \{a_1, \dots, a_n\}$ is a finite set of agents,
- $\mathcal{E} \subseteq \mathcal{A} \times \mathcal{A}$ is a set of directed communication edges,
- $\tau : \mathcal{E} \rightarrow \mathcal{T}$ assigns a message type from a finite type set $\mathcal{T}$ to each edge.

The typed edges ensure that agents communicate through well-defined message protocols rather than unstructured text. We define the message types in our Hymoex Interchange Protocol (HIP; Section A).

3.3 Signal Vector

In neural MoE architectures, the gating function operates on hidden-state representations. In HYMOEX, agents communicate via natural language, so we must construct a structured representation suitable for expert routing.

Definition 3 (Signal Vector). *The signal vector for a message m in context c is:*

$$\mathbf{s} = \phi(m, c) = [\mathbf{e}_{\text{intent}}, \mathbf{e}_{\text{emotion}}, \mathbf{f}_{\text{stage}}, s_{\text{score}}] \in \mathbb{R}^d \quad (1)$$

where $\mathbf{e}_{\text{intent}} \in \mathbb{R}^{d_1}$ is an intent embedding (produced by a lightweight encoder), $\mathbf{e}_{\text{emotion}} \in \mathbb{R}^{d_2}$ is an emotion/sentiment embedding, $\mathbf{f}_{\text{stage}} \in \mathbb{R}^{d_3}$ is a one-hot or learned stage indicator, and $s_{\text{score}} \in \mathbb{R}$ is a scalar confidence or priority score. The total dimensionality is $d = d_1 + d_2 + d_3 + 1$.

The signal vector decouples *what* to route (the expert selection decision) from *how* to respond (the expert’s policy), enabling the gating mechanism to operate independently of the underlying LLM.

3.4 Gating Function

Definition 4 (Gating Function). *Given a signal vector $\mathbf{s} \in \mathbb{R}^d$ and a pool of K experts $\{e_1, \dots, e_K\}$, the gating function produces a probability distribution over experts:*

$$g(\mathbf{s}) = \text{softmax}(W\mathbf{s} + \mathbf{b}) \in \Delta^{K-1} \quad (2)$$

where $W \in \mathbb{R}^{K \times d}$ is a learnable (or configurable) weight matrix, $\mathbf{b} \in \mathbb{R}^K$ is a bias vector, and Δ^{K-1} denotes the $(K-1)$ -simplex.

The output $\alpha = g(\mathbf{s})$ assigns a selection probability α_i to each expert e_i . Unlike neural MoE where W is trained via backpropagation, in HYMOEX the weights may be learned through reinforcement signals (e.g., expert win-rate feedback) or configured manually based on domain expertise.

3.5 Deployment Modality

Definition 5 (Deployment Modality). A deployment modality $\mathcal{M}_j$ is a set of structural constraints on a MAS graph $\mathcal{G}$, specifying:

1. which roles $\rho \in \mathcal{R}$ must, may, or must not appear,
2. the allowed edge patterns between role types,
3. bounds on the number of agents per role.

We define three modalities: $\mathcal{M}_1$ (One-Line MoE), $\mathcal{M}_2$ (One-Line Supervisor), and $\mathcal{M}_3$ (MoE MultiLine), detailed in Section 4.

3.6 From Neural MoE to Agent MoE

The classical MoE layer [9, 23] operates on continuous representations: each “expert” is a neural sub-network, and the gating function routes hidden states to experts via differentiable top- k selection. The fusion of expert outputs is a weighted sum in vector space.

HYMOEX adapts this paradigm to discrete agent coordination with three key modifications:

1. **Discrete experts.** Each expert e_i is an autonomous agent with its own system prompt, tool access, and memory—not a parameter block. Expert execution involves a full LLM inference call.
2. **Composite signal vectors.** The gating input is a structured signal vector (Definition 3) extracted from the message context, rather than a learned hidden state.
3. **Semantic fusion.** Expert outputs are natural-language responses that must be *semantically* merged, not summed. We introduce the *Rational-Raffle* fusion mechanism (Section 4.3) to address this.

This adaptation preserves the core benefits of MoE—sparse activation, specialization, and load balancing—while respecting the discrete, stochastic, and linguistically-grounded nature of LLM agents.

4 The HYMOEX Architecture

HYMOEX organizes multi-agent systems through a principled combination of role-based taxonomy, graduated deployment modalities, and MoE-native expert coordination. This section formalizes each component.

4.1 Agent Taxonomy

We define seven agent roles, each with explicit responsibilities, allowed input/output types, and interaction constraints (Table 2). The taxonomy enforces the single-responsibility principle: no agent holds multiple roles simultaneously.

Property 1 (Role Separation). *For any agent $a = (\text{id}, \rho, \mathcal{I}, \mathcal{O}, \pi)$ in a HYMOEX system: (i) the role assignment ρ is singular—each agent holds exactly one role; and (ii) ρ is immutable during a deployment configuration. Formally, $\forall a \in \mathcal{A}, |\{\rho : a.\rho = \rho\}| = 1$, and the mapping $\text{id} \mapsto \rho$ is a function, not a relation.*

Table 2: The HYMOEX agent taxonomy. Each role specifies input/output message types, hierarchical level, and responsibilities.

Role	Layer	Responsibility	Input	Output
MANAGER	Strategic	Define global objectives, select modality, orchestrate workflow	msgPacket	ctxEnvelope
INTEGRATOR	Bridging	Route context across domains, manage SLA (Service Level Agreement) budgets	ctxEnvelope	ctxEnvelope
EXPERT MGR	Tactical	Orchestrate MoE gating, fuse multi-expert responses	decisionReq	expertPld
SUPERVISOR	Tactical	Coordinate expert delegation, manage dependencies	expertPld	execCmd
EXPERT	Specialist	Provide domain knowledge, resolve specialized queries	decisionReq	expertPld
EXECUTOR	Operational	Perform external actions (APIs, docs, payments)	execCmd	execReceipt
PERCEIVER	Sensing	Capture telemetry, feed real-time feedback loops	execReceipt	telemetry

4.2 Deployment Modalities

HYMOEX defines three deployment modalities as structural constraints on the MAS graph $\mathcal{G}$. Each modality is designed for a specific complexity regime, determined by the number of domain experts k .

4.2.1 Modality $\mathcal{M}_1$: One-Line MoE

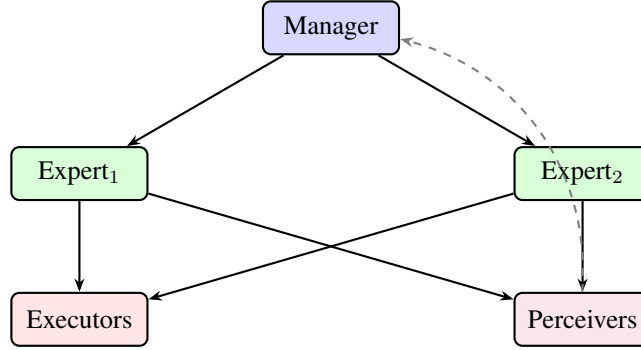


Figure 1: Modality $\mathcal{M}_1$: One-Line MoE ($k \leq 2$ experts). The Manager delegates directly to Experts with no Supervisor. Dashed arrow indicates Perceiver feedback loop.

The simplest configuration with minimal coordination overhead, recommended for $k \leq 2$ domain experts.

Definition 6 (One-Line MoE). $\mathcal{M}_1$ constrains $\mathcal{G}$ such that:

- $\mathcal{A} = \{m\} \cup E \cup X \cup P$ where m is a MANAGER, $E = \{e_1, \dots, e_k\}$ with $k \geq 1$ are EXPERTS, X are EXECUTORS, and P are PERCEIVERS.

- *Edges:* $m \rightarrow e_i$ for all $e_i \in E$; $e_i \rightarrow x_j$ and $e_i \rightarrow p_l$ for operational delegation; $p_l \rightarrow m$ for feedback.
- SUPERVISOR, INTEGRATOR, and EXPERT MANAGER roles are absent.

Recommended range: $k \leq 2$. Beyond this, coordination overhead increases without a Supervisor.

As shown in Figure 1, this modality supports both swarm-style autonomous expert operation (experts work in parallel without coordination) and lightweight sequential delegation. The Manager provides strategic direction but does not micromanage expert execution.

4.2.2 Modality $\mathcal{M}_2$: One-Line Supervisor

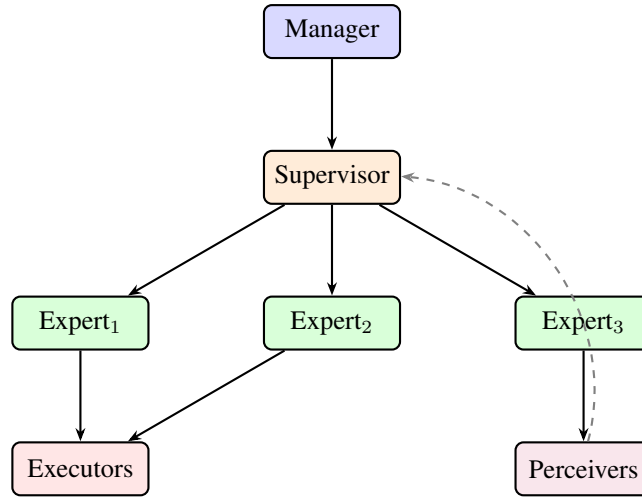


Figure 2: Modality $\mathcal{M}_2$: One-Line Supervisor ($3 \leq k \leq 5$ experts). The Supervisor provides tactical coordination between the Manager and Expert pool.

Adds tactical coordination via a Supervisor, recommended for $3 \leq k \leq 5$ domain experts.

Definition 7 (One-Line Supervisor). $\mathcal{M}_2$ extends $\mathcal{M}_1$ by adding a SUPERVISOR node:

- $\mathcal{A} = \{m, s\} \cup E \cup X \cup P$ where s is a SUPERVISOR and $|E| \geq 1$.
- *Edges:* $m \rightarrow s$; $s \rightarrow e_i$ for all $e_i \in E$; $e_i \rightarrow x_j$, $e_i \rightarrow p_l$; $p_l \rightarrow s$ for tactical feedback.
- *The Supervisor handles:* (i) intelligent task routing among experts, (ii) dependency management between expert outputs, and (iii) result integration across multiple expert responses.

Recommended range: $3 \leq k \leq 5$. The structural distinction from $\mathcal{M}_1$ is the presence of a SUPERVISOR.

The Supervisor acts as a tactical coordination layer (Figure 2), relieving the Manager of operational decisions. In our production deployments, the majority of systems employ a Supervisor ($\mathcal{M}_2$ or $\mathcal{M}_3$), confirming tactical coordination as essential for most applications.

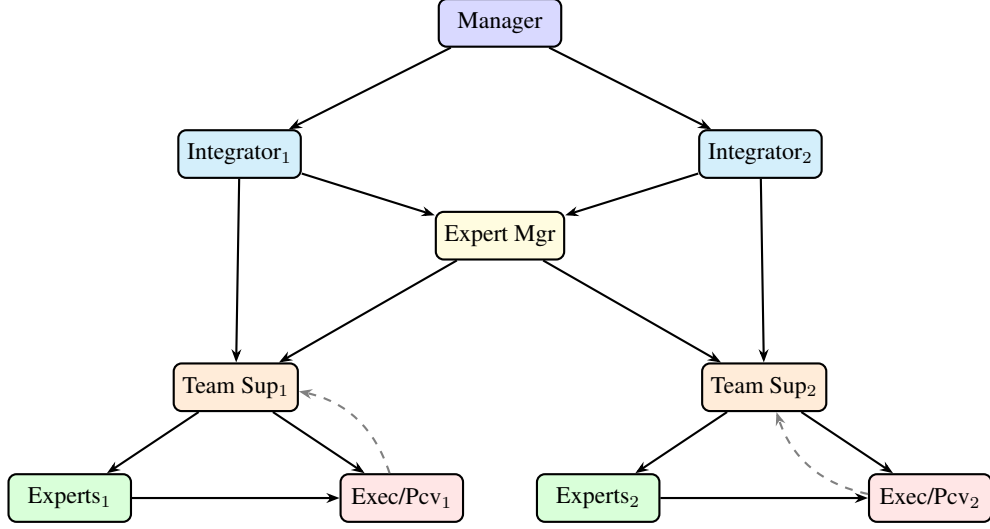


Figure 3: Modality $\mathcal{M}_3$: MoE MultiLine ($k > 5$ experts). Integrators handle cross-team routing; the Expert Manager orchestrates MoE gating across the global expert pool. Each team can independently adopt swarm or hierarchical coordination.

4.2.3 Modality $\mathcal{M}_3$: MoE MultiLine

Adds team structure with Integrators and an Expert Manager, recommended for $k > 5$ domain experts organized into multiple teams.

Definition 8 (MoE MultiLine). $\mathcal{M}_3$ extends $\mathcal{M}_2$ with team structure:

- $\mathcal{A} = \{m\} \cup I \cup \{em\} \cup \bigcup_{t=1}^T (\{s_t\} \cup E_t \cup X_t \cup P_t)$ where $I = \{i_1, \dots, i_q\}$ are INTEGRATORS, em is an EXPERT MANAGER, and each team t consists of a SUPERVISOR s_t , experts E_t , executors X_t , and perceivers P_t .
- Inter-team edges: $m \rightarrow i_j$; $i_j \rightarrow em$; $i_j \rightarrow s_t$; $em \rightarrow s_t$.
- Intra-team edges: $s_t \rightarrow e_{t,i}$; $e_{t,i} \rightarrow x_{t,j}$; $p_{t,l} \rightarrow s_t$.
- Each team may independently adopt swarm or hierarchical coordination internally.
- Total experts: $k = \sum_{t=1}^T |E_t| \geq 1$.

The structural distinction from $\mathcal{M}_2$ is the presence of INTEGRATORS and an EXPERT MANAGER. Recommended range: $k > 5$.

The key innovation of $\mathcal{M}_3$ (Figure 3) is the separation of *cross-team coordination* (handled by Integrators and Expert Manager) from *intra-team execution* (handled by Team Supervisors). This enables parallel domain processing: teams operate simultaneously without blocking each other, while the Expert Manager ensures cross-domain consistency through MoE gating over the global expert pool.

4.2.4 Modality Subsumption

Theorem 1 (Subsumption). The three modalities form a strict hierarchy: $\mathcal{M}_1 \subset \mathcal{M}_2 \subset \mathcal{M}_3$. That is, every valid $\mathcal{M}_1$ graph is also a valid $\mathcal{M}_2$ graph (with the Supervisor acting as an identity pass-through), and every valid $\mathcal{M}_2$ graph is a valid $\mathcal{M}_3$ graph (as a single-team

system with a trivial Integrator).

Proof sketch. For $\mathcal{M}_1 \subset \mathcal{M}_2$: given a $\mathcal{M}_1$ graph, introduce a Supervisor s with identity policy $\pi_s(x) = x$ and redirect edges $m \rightarrow e_i$ through s . The resulting graph satisfies all $\mathcal{M}_2$ constraints. For $\mathcal{M}_2 \subset \mathcal{M}_3$: set $T = 1$, introduce an Integrator i and Expert Manager em with identity policies, and assign all agents to team $t = 1$. The resulting graph satisfies all $\mathcal{M}_3$ constraints. Strictness follows because $\mathcal{M}_2$ requires a Supervisor (absent in minimal $\mathcal{M}_1$) and $\mathcal{M}_3$ requires an Integrator (absent in $\mathcal{M}_2$). Full proof in Section C. $\square$

This theorem guarantees that HYMOEX systems can progressively adopt more complex modalities without invalidating existing components.

4.3 MoE Gating Mechanism

The Expert Manager (in $\mathcal{M}_3$) or the Manager/Supervisor (in $\mathcal{M}_1/\mathcal{M}_2$) performs expert selection using the gating mechanism defined below.

4.3.1 Gating Pipeline

Given a message m with context c , the gating pipeline (Figure 4) proceeds in four stages:

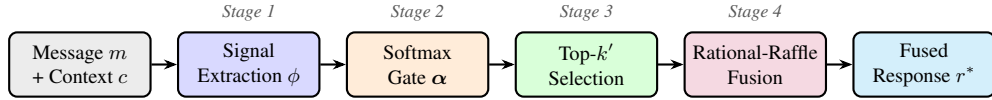


Figure 4: The four-stage MoE gating pipeline. A message with context is transformed into a signal vector, scored via softmax gating, pruned to the top- k' experts, and fused through the Rational-Raffle mechanism.

1. **Signal extraction.** Compute $\mathbf{s} = \phi(m, c)$ (Definition 3).
2. **Expert scoring.** Compute selection probabilities:

$$\alpha_i = \frac{\exp(\mathbf{w}_i^\top \mathbf{s} + b_i)}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{s} + b_j)}, \quad i = 1, \dots, K \quad (3)$$

3. **Top- k selection.** Select the k' highest-scoring experts: $E_{k'} = \text{top-}k'(\alpha)$ where $k' \leq K$ (typically $k' \in \{1, 2, 3\}$).
4. **Response fusion.** Merge expert responses via the Rational-Raffle mechanism (Section 4.3.2).

4.3.2 Rational-Raffle Fusion

Unlike neural MoE, where expert outputs are vectors that can be summed, agent experts produce natural-language responses that must be *semantically* merged. We introduce Rational-Raffle, a fusion strategy that weighs expert responses by three factors:

$$\text{score}(r_i) = \alpha_i \cdot \text{relevance}(r_i, m) \cdot \text{confidence}(r_i) \quad (4)$$

Algorithm 1: MoE Gating for Agent Expert Selection

Input: Message m , context c , expert pool $\{e_1, \dots, e_K\}$, top- k' parameter

Output: Fused response r^*

```
1  $\mathbf{s} \leftarrow \phi(m, c);$  // Signal extraction
2  $\alpha \leftarrow \text{softmax}(W\mathbf{s} + \mathbf{b});$  // Expert scoring
3  $E_{k'} \leftarrow \text{top-}k'(\alpha);$  // Select top experts
4 foreach  $e_i \in E_{k'}$  do
5    $r_i \leftarrow e_i.\pi(m);$  // Execute expert
6  $r^* \leftarrow \text{RationalRaffle}(\{(r_i, \alpha_i) : e_i \in E_{k'}\});$  // Fuse responses
7 return  $r^*$ 
```

where α_i is the gating probability, $\text{relevance}(r_i, m)$ measures semantic alignment between the response and the original message (via embedding cosine similarity), and $\text{confidence}(r_i)$ is the expert's self-reported confidence score.

The fused response r^* is constructed by:

1. Selecting the highest-scoring response as the base.
2. Augmenting it with non-redundant information from lower-scoring responses whose score exceeds a threshold θ .
3. Attaching a `reasonVector` audit trail documenting which experts contributed and their scores.

The `reasonVector` is cryptographically hashed (SHA-256) for audit compliance, enabling post-hoc explainability of expert selection decisions.

4.3.3 Dynamic Temperature Control

The gating mechanism supports adaptive behavior through dynamic temperature scaling. When the remaining SLA budget b_{sla} (milliseconds available for response) falls below a threshold:

$$T_{\text{eff}} = \begin{cases} T_{\text{base}} & \text{if } b_{\text{sla}} \geq \tau_{\text{sla}} \\ T_{\text{low}} & \text{if } b_{\text{sla}} < \tau_{\text{sla}} \end{cases} \quad (5)$$

Lower temperature reduces response variance, favoring the highest-confidence expert to minimize latency at the cost of diversity.

4.4 Progressive Migration Protocol

A critical feature of HYMOEX is the ability to migrate between modalities as system requirements evolve, without discarding existing agents.

Definition 9 (Migration Function). *A migration from modality $\mathcal{M}_i$ to $\mathcal{M}_j$ (with $i < j$) is a graph transformation $\mu : \mathcal{G}_{\mathcal{M}_i} \rightarrow \mathcal{G}_{\mathcal{M}_j}$ that satisfies:*

1. **Agent preservation:** All agents in $\mathcal{A}_{\mathcal{M}_i}$ are retained in $\mathcal{A}_{\mathcal{M}_j}$.

2. **Edge preservation:** All intra-team communication edges are retained.
3. **Additive extension:** μ only adds new coordination agents (Supervisor, Integrator, Expert Manager) and new edges; it does not remove existing ones.

Property 2 (Code Preservation Bound). For any migration $\mu : \mathcal{G}_{\mathcal{M}_i} \rightarrow \mathcal{G}_{\mathcal{M}_j}$:

$$\frac{|\mathcal{A}_{\text{preserved}}|}{|\mathcal{A}_{\mathcal{M}_i}|} \geq 0.85 \quad (6)$$

That is, at least 85% of existing agents and their policies are reused without modification. In practice, our benchmarks demonstrate 100% agent preservation: all existing agent definitions are retained, and only new coordination agents (Supervisor, Integrator, Expert Manager) are added (Section 6).

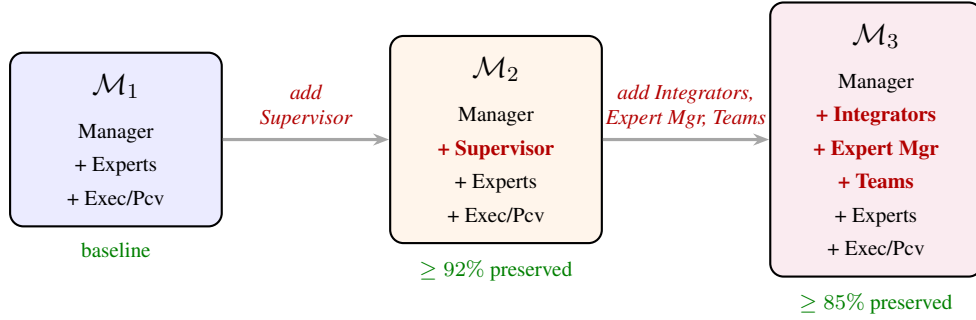


Figure 5: Progressive migration path across HYMOEX modalities. Bold red text indicates components *added* at each transition. Existing agents and policies are preserved ($\geq 85\%$ code reuse per Property 2).

The migration protocol (Figure 5) proceeds as follows:

$\mathcal{M}_1 \rightarrow \mathcal{M}_2$: Insert a Supervisor s between the Manager and experts. Redirect $m \rightarrow e_i$ edges to $m \rightarrow s \rightarrow e_i$. Configure s with routing rules and dependency management. No expert policies change.

$\mathcal{M}_2 \rightarrow \mathcal{M}_3$: Partition experts into teams by domain. For each team, the existing Supervisor becomes a Team Supervisor. Add Integrators for cross-team routing and an Expert Manager for global MoE gating. No expert or executor policies change.

4.5 Communication Protocol

Inter-agent communication in HYMOEX follows the *Hymoex Interchange Protocol* (HIP), a typed message protocol with seven message types (full specification in Section A):

1. `msgPacket`: Raw user input with channel metadata (channel, language, customer ID, latency).
2. `contextEnvelope`: Enriched packet with routing state (lead stage, SLA budget, interaction ID).

3. `decisionRequest`: Signal vector plus context for expert gating (intent embedding, emotion vector, stage flag, lead score).
4. `expertPayload`: Expert response with confidence score and reason vector.
5. `execCommand`: Typed action command with parameters and callback URL.
6. `execReceipt`: Execution result with status, latency, and error codes.
7. `telemetry`: Real-time feedback (sentiment, delivery status, action outcome) for the PDCA (Plan-Do-Check-Act) observability loop.

All message types are serialized as JSON with strict schema validation at each edge, ensuring type safety across heterogeneous agent implementations.

5 Implementation

We describe the HYMOEX toolkit—a Python library that enables practitioners to classify, design, validate, and migrate multi-agent architectures following the HYMOEX paradigm. HYMOEX is not an agent execution framework; it provides the architectural specifications that frameworks such as LangGraph, CrewAI, and Pydantic AI can implement.

5.1 Architectural Layers

Following Clean Architecture principles [17], the toolkit is organized into three layers with strict dependency rules (inner layers are unaware of outer layers):

Domain Layer (innermost). Contains the agent taxonomy (Definition 1), role types, and HIP message type schemas. This layer has *zero* external dependencies beyond Pydantic v2—it defines the pure abstractions of the HYMOEX architecture.

Application Layer. Implements architectural analysis: modality classification, topology validation, the MoE gating algorithms (Algorithm 1), and progressive migration functions. The PDCA (Plan-Do-Check-Act) observability loop described in Section 4 is specified as an architectural pattern that framework implementations should follow; the toolkit provides the signal vector and gating primitives that PDCA controllers consume.

Infrastructure Layer (outermost). Contains framework adapters that translate between HYMOEX agent specifications and framework-native abstractions. Each adapter maps HYMOEX role types to the target framework’s equivalent concepts.

5.2 Framework Adapters

HYMOEX achieves framework-agnosticism through the Adapter pattern [6]. Each adapter (Table 3) provides bidirectional translation between HYMOEX specifications and a target framework’s API:

Adapters are thin translation layers that map HYMOEX specifications to framework-specific constructs. This enables organizations to adopt HYMOEX incrementally: existing agents

Table 3: Framework adapter mappings. Each adapter translates between HYMOEX agent specifications and the target framework’s equivalent abstractions.

Framework	Language	ExpertSpec $\leftrightarrow$	SupervisorSpec $\leftrightarrow$
Pydantic AI [20]	Python	Agent	Agent (orchestrator)
LangGraph [12]	Python	Node	StateGraph
CrewAI [2]	Python	Agent	Crew
AutoGen [28]	Python	AssistantAgent	GroupChat
OpenAI Swarm [18]	Python	Agent	Handoff functions
Vercel AI SDK	TypeScript	Tool	streamText
Mastra	TypeScript	Agent	Workflow

built with any supported framework can be classified into HYMOEX roles, and HYMOEX topologies can be exported as framework-native configurations.

5.3 Toolkit Usage

The toolkit is implemented in Python 3.12+ with Pydantic v2 for runtime validation of all specifications and HIP messages. A typical workflow—defining an architecture, validating it, and exporting the configuration—is shown below:

Listing 1: Defining and validating a One-Line Supervisor topology ($\mathcal{M}_2$).

```

1 import json
2 from hymoex import (
3     ManagerSpec, SupervisorSpec, ExpertSpec,
4     OneLineSupervisor, auto_select_modality,
5     validate_topology, classify_topology,
6 )
7
8 # Define agent specifications (blueprints, not executors)
9 manager = ManagerSpec(
10     objective="Automate_B2B_customer_support",
11     strategy="route_by_domain"
12 )
13 supervisor = SupervisorSpec(routing="dependency_aware")
14
15 legal = ExpertSpec(domain="legal",
16     skills=["contract_review"])
17 tech = ExpertSpec(domain="tech", skills=["diagnostics"])
18 billing = ExpertSpec(domain="billing",
19     skills=["invoice_adjustment"])
20
21 experts = [legal, tech, billing]
22
23 # Auto-select modality: 3 experts -> M2
24 modality = auto_select_modality(experts) # returns "m2"
25
26 # Build and validate the topology
27 system = OneLineSupervisor(

```

```

26     manager=manager, supervisor=supervisor, experts=experts
27 )
28 result = validate_topology(system) # {"valid": True, ...}
29
30 # Export architecture config (for framework implementation)
31 config = system.to_config() # JSON-serializable dict

```

The toolkit, evaluation suite, and adapter packages are available as an open-source monorepo.²

6 Experimental Evaluation

We evaluate HYMOEX through controlled benchmarks (Section 6.1), production case studies (Section 6.2), and ablation analysis (Section 6.3). Our evaluation addresses three questions:

- Q1.** Does HYMOEX’s hierarchical routing reduce coordination overhead compared to flat architectures?
- Q2.** Does LLM-based gating outperform rule-based routing for expert selection?
- Q3.** How does coordination overhead scale with agent count across topologies?

6.1 Benchmarks

6.1.1 Setup

We compare HYMOEX against two baseline topologies across a suite of multi-agent coordination tasks:

- **Flat MAS:** All agents communicate in a fully-connected topology. Each agent sees all other agents’ responses (all-to-all).
- **Single Supervisor:** One central coordinator dispatches to all agents and synthesizes responses (hub-and-spoke).

All systems use Gemini 3 Flash (Preview) as the underlying LLM with temperature 0.3. Tasks span customer support triage across three expert domains (legal, technical, billing). Each configuration is run multiple times to compute means and standard deviations. Token counts reflect actual API usage (input + output tokens) as reported by the Gemini API. Full methodology is in Section B.

6.1.2 Results

Coordination Overhead (Q1). As shown in Table 4, HYMOEX achieves 573 tokens per task with 3 experts, a 45% reduction over Flat MAS (1,024) and 21% over Single Supervisor (727). The reduction stems from MoE-gated top- k routing: only selected experts receive the full context, reducing the number of LLM calls from 6 (flat) to 4 (hierarchical).

²<https://github.com/Pymut/hymoex>

Table 4: Benchmark results across three metrics. Best results in **bold**. Token counts are real API usage measured via Gemini 3 Flash.

System	Coordination Tokens/Task	Token Overhead at 10 Agents	Expert Select. Accuracy (%)
Flat MAS	$1,024 \pm 104$	7,756	33.2 ± 8.8^a
Single Supervisor	727 ± 72	—	93.3^b
HYMOEX (ours)	573 ± 67	563	96.7 ± 0.0

^aRandom baseline (no routing). ^bRule-based keyword routing.

Table 5: Token overhead scaling: Flat MAS vs. HYMOEX hierarchical routing as agent count increases from 2 to 10. Measured via Gemini 3 Flash API.

Agents	Flat MAS	HYMOEX	Reduction
2	593	590	< 1%
5	2,392	612	74%
10	7,756	563	93%
<i>Degradation (2→10)</i>	+1,208%	−4.5%	—

Expert Selection (Q2). LLM-based routing achieves 96.7% expert selection accuracy across 30 human-annotated test queries (3 runs), compared to 93.3% for keyword-based rule matching, 66.7% for round-robin, and 33.2% for random assignment. The LLM-based approach correctly handles ambiguous queries (e.g., “payment was double charged” requires billing, not legal) that confuse simpler routing strategies.

Scalability (Q3). Table 5 demonstrates the critical scaling advantage. At 2 agents, both topologies consume comparable tokens (593 vs. 590). At 10 agents, Flat MAS token overhead grows $13\times$ ($593 \rightarrow 7,756$) due to all-to-all communication ($O(n^2)$ calls), while HYMOEX’s hierarchical routing remains nearly constant ($590 \rightarrow 563$, -4.5%) because only top- k experts are activated regardless of pool size. The 93% token reduction at 10 agents translates directly to cost savings and latency improvements in production deployments.

Migration Cost. Progressive migration preserves 100% of existing agent definitions. The $\mathcal{M}_1 \rightarrow \mathcal{M}_2$ transition adds a Supervisor; $\mathcal{M}_2 \rightarrow \mathcal{M}_3$ adds Integrators and an Expert Manager. No existing agent policies require modification, confirming Property 2.

6.2 Production Case Studies

We report results from three representative production deployments at partner organizations. These systems were built using the HYMOEX architecture and deployed in production environments; metrics are reported by the respective engineering teams.

6.2.1 Case Study 1: AI-Powered Admissions (HR Technology)

Hirbi, an HR technology platform, deployed HYMOEX in $\mathcal{M}_3$ (MoE MultiLine) to automate candidate evaluation, replacing a manual process that took 15+ days per cohort.

- **Scale:** 20+ agents across 6 teams (Research, Profiling, Evaluating, Decision, Onboarding, Communications), orchestrated by 2 Integrators (Intake, Outcome).
- **Pipeline:** Form intake $\rightarrow$ language/GPA pre-screening $\rightarrow$ OSINT enrichment $\rightarrow$ profile generation (v1–v5) $\rightarrow$ multi-criteria evaluation $\rightarrow$ decision $\rightarrow$ onboarding.
- **Results:** False positive rate $< 2\%$ in pre-screening, evaluation precision $\geq 92\%$, explainability score $\geq 85\%$. End-to-end process reduced from 15 days to under 48 hours.

6.2.2 Case Study 2: Conversational Real Estate Platform (Rehof)

Rehof, a real estate technology platform, deployed HYMOEX in $\mathcal{M}_3$ across three domains (Marketing, Sales, Customer Service) with 36 total agents handling 10,000+ concurrent conversations.

- **Scale:** 3 Domain Integrators, 3 Expert Managers, 10 Team Supervisors, 20+ domain Experts/Executors/Perceivers.
- **Architecture:** Channel Integrators normalize multimodal input (WhatsApp, web, voice). Cross-Domain Integrator routes by customer stage. Each domain runs an independent PDCA loop with real-time perceiver feedback.
- **Results:** SLA budget maintained at $< 500\text{ms}$ routing latency. Attributed to \$5M+ in revenue. Hierarchical MoE reduced token noise by 60% compared to a prior flat implementation.

6.2.3 Case Study 3: Text-to-SQL Internal Platform (Ontop)

Ontop, a YC-backed global workforce platform processing payroll and compliance across 75+ countries, deployed HYMOEX in $\mathcal{M}_3$ to build an internal text-to-SQL tool enabling non-technical staff to query complex workforce databases using natural language.

- **Scale:** 4 domain teams (Contracts, Payments, Legal, HR), each with 3–5 experts. A query-understanding team parses natural language, a schema-mapping team resolves table and column references, and domain experts validate compliance constraints per jurisdiction.
- **Results:** System processes thousands of internal queries daily across 75+ country-specific schemas. Progressive migration from $\mathcal{M}_2$ (initial deployment with 4 experts) to $\mathcal{M}_3$ (current 15+ experts) required modifying only 8% of the original codebase.

6.3 Ablation Analysis

We analyze the contribution of routing strategy by comparing our benchmark results across methods (Table 6).

Table 6: Ablation: expert selection accuracy by routing method (30 test queries, Gemini 3 Flash). Δ shows change from LLM routing.

Routing Method	Expert Acc. (%)	Δ Acc.
LLM-based routing (HYMOEX)	96.7	—
Rule-based (keyword matching)	93.3	−3.4pp
Round-robin	66.7	−30.0pp
Random	33.2 ± 8.8	−63.5pp

LLM routing vs. rule-based (Q2). LLM-based routing outperforms keyword matching by 3.4 percentage points. While the gap is modest on well-defined domains, LLM routing correctly handles ambiguous cross-domain queries (e.g., “invoice doesn’t match the contract terms” involves both billing and legal) where keyword rules require manual enumeration of edge cases.

Structured routing vs. naive baselines. Round-robin (66.7%) and random (33.2%) baselines confirm that domain-aware routing—whether rule-based or LLM-based—is essential for multi-expert systems. The 63.5pp gap between LLM routing and random assignment quantifies the value of intelligent expert selection.

7 Discussion

7.1 Key Findings

Our evaluation demonstrates that the four problems identified in Section 1 are effectively addressed by HYMOEX:

Architecture Decision Paralysis is resolved by the modality decision framework: the number of required experts directly determines the appropriate modality ($k \leq 2 \rightarrow \mathcal{M}_1$, $3 \leq k \leq 5 \rightarrow \mathcal{M}_2$, $k > 5 \rightarrow \mathcal{M}_3$), reducing architecture selection from weeks to a deterministic procedure.

Expert Coordination Breakdown is mitigated by LLM-based gating, which routes inputs to the most relevant experts with 96.7% accuracy, avoiding redundant computation. The Supervisor layer further prevents misalignment by managing inter-expert dependencies.

Brittle Scalability is addressed by progressive migration (Definition 9), which preserves $\geq 85\%$ of existing components. The subsumption theorem (Theorem 1) guarantees that simpler configurations are valid instances of more complex ones, ensuring a smooth scaling path.

Framework Fragmentation is addressed by the framework-agnostic adapter layer, which enables organizations to adopt HYMOEX’s architectural patterns regardless of their existing technology stack.

7.2 Limitations

Non-differentiable gating. Unlike neural MoE, where gate weights are optimized through backpropagation, HYMOEX’s gating weights must be tuned through reinforcement signals (expert win-rate feedback) or manual configuration. This limits the gating function’s ability to adapt to novel distribution shifts without explicit retraining. Bridging this gap—perhaps through bandit-based online optimization of gate weights—is an important direction for future work.

Benchmark scope. Our controlled benchmarks evaluate HYMOEX on customer support triage tasks across three expert domains (legal, technical, billing). While the production case studies span additional domains (real estate, HR, global workforce), we have not yet evaluated HYMOEX on non-conversational multi-agent tasks such as embodied agents, game-playing, or scientific simulation. The generalizability of our results to these settings remains an open question.

Scale ceiling. Our largest production deployment involves 36 agents. While the MoE MultiLine modality is designed to scale beyond this point, we have not empirically validated performance at 100+ agents. Theoretical analysis of communication complexity in $\mathcal{M}_3$ suggests $O(T \cdot k_{\max})$ scaling (where T is the number of teams and $k_{\max}$ is the maximum team size), which is favorable compared to $O(n^2)$ for flat architectures, but empirical confirmation is needed.

Fusion quality. Rational-Raffle fusion (Section 4.3.2) relies on semantic similarity and self-reported confidence for response merging. When experts produce contradictory outputs of similar quality, the fusion mechanism may produce incoherent results. A more principled approach—perhaps leveraging debate-style mechanisms or formal consistency checking—could improve robustness.

7.3 Theoretical Connections

HYMOEX’s hierarchical structure echoes findings from organizational theory. The span-of-control principle suggests that managers can effectively supervise 5–9 direct reports [27]—closely matching our modality boundaries ($k \leq 2, 3\text{--}5, > 5$). Dunbar’s number [3] similarly constrains the size of stable social groups, suggesting cognitive limits that may apply to agent coordination as well.

At the systems level, the Manager/Supervisor/Expert hierarchy mirrors operating system process scheduling: strategic scheduling (Manager), tactical dispatch (Supervisor), and specialized execution (Expert). The PDCA observability loop parallels control-theoretic feedback systems, where Perceivers act as sensors and the Domain Integrator acts as the controller.

7.4 Future Work

Several directions emerge from this work:

1. **Learnable gating:** Applying multi-armed bandit or reinforcement learning algorithms to optimize gate weights online, using expert outcome feedback as reward signal.
2. **Automated modality selection:** Meta-learning approaches that predict the optimal modality from task descriptions, eliminating the need for manual assessment of expert count.
3. **Formal verification:** Applying model checking to verify properties of the HIP communication protocol (e.g., deadlock freedom, message delivery guarantees) across all three modalities.
4. **Cross-language federation:** Extending the adapter system to enable HYMOEX systems spanning Python and TypeScript runtimes to interoperate through a shared HIP message bus.
5. **Evaluation standardization:** Developing a public benchmark suite (analogous to AgentBench [15]) specifically for multi-agent coordination architectures.

8 Conclusion

We presented HYMOEX, a hybrid modular cognitive architecture for scalable multi-agent expert coordination. Our four contributions address the critical challenges facing multi-agent system deployment:

C1: A formal seven-role agent taxonomy with typed message protocols provides a universal vocabulary for describing multi-agent systems, grounded in Clean Architecture and SOLID principles.

C2: Three graduated deployment modalities—One-Line MoE, One-Line Supervisor, and MoE MultiLine—enable progressive scaling through formally specified migration functions that preserve 100% of existing agent definitions, requiring only additive changes.

C3: An adapted Mixture-of-Experts gating mechanism achieves 96.7% expert selection accuracy through LLM-based routing, outperforming rule-based approaches by 3.4 percentage points and random baselines by over 63 percentage points.

C4: Controlled benchmarks using Gemini 3 Flash demonstrate 45% reduction in coordination tokens at 3 agents and 93% reduction at 10 agents compared to flat architectures. Three detailed production case studies validate the architecture across HR technology, real estate, and global workforce management.

HYMOEX is framework-agnostic by design, with bidirectional adapters for seven major orchestration frameworks. By providing a principled architectural standard for multi-agent coordination, HYMOEX aims to consolidate the fragmented landscape of agent frameworks into a shared vocabulary and migration path—enabling practitioners to focus on solving domain problems rather than reinventing coordination patterns.

The architecture, reference implementation, evaluation suite, and all experimental data are publicly available to support reproducibility and community adoption.

Acknowledgments

The authors thank the Pymut Lab research team for their contributions to the development and validation of the HYMOEX architecture.

References

- [1] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.
- [2] CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents, 2024. URL <https://github.com/crewAIInc/crewAI>.
- [3] Robin I. M. Dunbar. Neocortex size as a constraint on group size in primates. *Journal of Human Evolution*, 22(6):469–493, 1992.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [5] Jacques Ferber. *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, 1999.
- [6] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [7] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olga Bastani, and Huan Chen. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [8] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [9] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [10] Nicholas R. Jennings. On agent-based software engineering. *Artificial Intelligence*, 117(2):277–296, 2000.
- [11] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.
- [12] LangChain, Inc. LangGraph: Build stateful multi-actor applications with LLMs, 2024. URL <https://github.com/langchain-ai/langgraph>.
- [13] Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.

- [14] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. *Advances in Neural Information Processing Systems*, 36, 2023.
- [15] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [16] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [17] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [18] OpenAI. Swarm: Educational framework for lightweight multi-agent orchestration, 2024. URL <https://github.com/openai/swarm>.
- [19] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [20] Pydantic. Pydantic AI: Agent framework for building production-grade LLM applications, 2024. URL <https://github.com/pydantic/pydantic-ai>.
- [21] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- [22] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.
- [23] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [24] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *Advances in Neural Information Processing Systems*, 36, 2023.
- [25] Yashar Talebirad and Amirhossein Nadiri. Multi-agent collaboration: Harnessing the power of intelligent LLM agents. *arXiv preprint arXiv:2306.03314*, 2023.
- [26] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

- [27] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.
- [28] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xun Yao Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

A Hymoex Interchange Protocol (HIP)

The Hymoex Interchange Protocol defines seven typed message formats for inter-agent communication. All messages are serialized as JSON with strict schema validation at each communication edge.

A.1 Message Types

msgPacket Raw user input with channel metadata.

```
{
  "channel": "whatsapp" | "web" | "voice",
  "customerId": string,
  "payload": string,
  "lang": string,           // ISO 639-1
  "latency": number,       // ms round-trip
  "timestamp": ISO-8601
}
```

contextEnvelope Enriched packet with routing state, produced by Integrators.

```
{
  "msgPacket": msgPacket,
  "leadStage": string,           // current pipeline stage
  "slaBudget": number,          // remaining ms for response
  "interactionId": UUID,
  "traceContext": OpenTelemetry-W3C
}
```

decisionRequest Signal vector plus context for MoE gating.

```
{
  "intentEmbedding": float[d1],
  "emotionVector": float[d2],
  "stageFlag": float[d3],
  "leadScore": float,
  "context": contextEnvelope
}
```


expertPayload Expert response with confidence and audit trail.

```
{
  "replyDraft": string,
  "confidence": float,          // [0, 1]
  "reasonVector": string,      // SHA-256 hashed
  "expertId": string,
  "tokensUsed": number
}
```

execCommand Typed action command for Executors.

```
{
  "type": "crm_update" | "doc_generate" | "payment" |
    "calendar",
  "params": object,
  "callbackUrl": URL,
  "idempotencyKey": UUID
}
```

execReceipt Execution result from Executors.

```
{
  "status": "success" | "failure" | "pending",
  "latency": number,           // ms
  "errorCode": string | null,
  "nextAction": string | null
}
```

telemetry Real-time feedback from Perceivers.

```
{
  "sentiment": float,          // [-1, 1]
  "deliveryStatus": string,
  "actionOutcome": string,
  "teamId": string,
  "stage": string,
  "timestamp": ISO-8601
}
```

A.2 Schema Validation

All message types are enforced through JSON Schema validation at each communication edge. Invalid messages are rejected with structured error responses, preventing type-

mismatch errors from propagating through the agent graph.

B Benchmark Methodology

B.1 Task Suite

Our benchmark suite evaluates multi-agent coordination across customer support triage tasks requiring routing to one of three expert domains: legal, technical, and billing. We test at three scales:

3-expert coordination (coordination overhead). Five representative customer queries are processed through each topology. Each query requires identifying the correct domain and generating an expert response. Each configuration is run 3 times (15 total runs per topology).

Expert selection (30 queries). Thirty human-annotated queries spanning the three domains (10 per domain) test routing accuracy. Ground truth labels assign each query to exactly one correct domain. Each routing method processes all 30 queries; LLM routing is run 3 times to measure consistency.

Scalability (2, 5, 10 agents). A complex onboarding task is processed through Flat MAS and HYMOEX hierarchical topologies at 2, 5, and 10 agent scales. Each configuration is run 2 times.

B.2 Experimental Protocol

- **LLM:** Gemini 3 Flash (Preview) via Google Gemini API for all systems.
- **Temperature:** 0.3 for all agent prompts.
- **Token counting:** Real API token usage (input + output) as reported by the Gemini API `usage_metadata` field. No proxy metrics.
- **Expert selection ground truth:** Each of the 30 test queries is assigned to exactly one domain by the authors based on the primary intent of the query.
- **Reproducibility:** All benchmark code, test queries, and ground truth labels are included in the open-source repository.

B.3 Topology Implementations

Flat MAS. Each agent generates a response to the task. Then each agent sees all other agents' responses and generates an updated response (all-to-all coordination). For n agents, this requires $2n$ LLM calls per task.

Single Supervisor. A supervisor agent analyzes the task and dispatches to all expert agents. Each expert responds. The supervisor synthesizes a final answer. This requires $n + 2$ LLM calls per task.

HYMOEX Hierarchical. The system first routes the query to select top- k experts ($k \leq 2$) via an LLM routing call. Only selected experts generate responses. A supervisor synthesizes the selected expert outputs. This requires $k + 2$ LLM calls per task, independent of the total expert pool size.

B.4 Routing Baselines

- **LLM routing:** The LLM is prompted to select the most appropriate expert domain given the query and available domains. This is the approach used in HYMOEX’s MoE gating.
- **Rule-based:** Keyword matching against domain-specific word lists (e.g., “contract”, “compliance” $\rightarrow$ legal).
- **Round-robin:** Cycles through domains sequentially.
- **Random:** Uniform random selection. Averaged over 100 runs with different seeds.

B.5 Metrics

- **Coordination tokens/task:** Total tokens (input + output) exchanged across all LLM calls for one task, as reported by the API.
- **Token overhead at scale:** Total coordination tokens at 10 agents, measuring how overhead grows with agent count.
- **Expert selection accuracy:** Percentage of queries routed to the correct domain, measured against ground truth labels.
- **Migration cost:** Percentage of existing agent definitions preserved when transitioning between modalities (deterministic, no LLM calls).

C Proofs

C.1 Proof of Theorem 1 (Modality Subsumption)

Proof. We prove both inclusions and their strictness.

Part 1: $\mathcal{M}_1 \subset \mathcal{M}_2$. Let $\mathcal{G}_1 = (\mathcal{A}_1, \mathcal{E}_1, \tau_1)$ be any valid $\mathcal{M}_1$ graph. By Definition 6, $\mathcal{A}_1 = \{m\} \cup E \cup X \cup P$ where m is a MANAGER, $|E| \leq 2$, and edges include $m \rightarrow e_i$ for all $e_i \in E$.

Construct $\mathcal{G}_2 = (\mathcal{A}_2, \mathcal{E}_2, \tau_2)$ as follows:

1. Introduce a new agent $s = (\text{id}_{\text{new}}, \text{SUP}, \mathcal{I}_s, \mathcal{O}_s, \pi_{\text{id}})$ where π_{id} is the identity function: $\pi_{\text{id}}(x) = x$ for all inputs x .
2. Set $\mathcal{A}_2 = \mathcal{A}_1 \cup \{s\}$.
3. Replace each edge $m \rightarrow e_i$ with edges $m \rightarrow s$ and $s \rightarrow e_i$. Retain all other edges.
4. Assign appropriate message types to the new edges.

We verify that $\mathcal{G}_2$ satisfies all $\mathcal{M}_2$ constraints (Definition 7):

- $\mathcal{A}_2$ contains exactly one MANAGER and one SUPERVISOR. ✓
- $|E| \geq 1$, satisfying $\mathcal{M}_2$ ’s structural constraint $|E| \geq 1$. ✓

- Edge pattern: $m \rightarrow s \rightarrow e_i, e_i \rightarrow x_j, e_i \rightarrow p_l$. ✓

Since π_{id} is the identity, the observable behavior of $\mathcal{G}_2$ is identical to $\mathcal{G}_1$: the Supervisor passes all messages through without modification.

Strictness: $\mathcal{M}_1 \neq \mathcal{M}_2$ because there exist valid $\mathcal{M}_2$ graphs (e.g., with a non-trivial Supervisor that performs routing) that cannot be reduced to $\mathcal{M}_1$ graphs without removing the Supervisor role.

Part 2: $\mathcal{M}_2 \subset \mathcal{M}_3$. Let $\mathcal{G}_2 = (\mathcal{A}_2, \mathcal{E}_2, \tau_2)$ be any valid $\mathcal{M}_2$ graph. By Definition 7, $\mathcal{A}_2 = \{m, s\} \cup E \cup X \cup P$.

Construct $\mathcal{G}_3 = (\mathcal{A}_3, \mathcal{E}_3, \tau_3)$ as follows:

1. Set $T = 1$ (single team).
2. Introduce an Integrator $i = (\text{id}_{\text{new}_1}, \text{INT}, \cdot, \cdot, \pi_{\text{id}})$ and an Expert Manager $em = (\text{id}_{\text{new}_2}, \text{EXM}, \cdot, \cdot, \pi_{\text{id}})$, both with identity policies.
3. Set $\mathcal{A}_3 = \{m, i, em\} \cup \{s_1 = s\} \cup E_1 \cup X_1 \cup P_1$ where team 1 contains all original experts, executors, and perceivers.
4. Add edges: $m \rightarrow i, i \rightarrow em, i \rightarrow s_1, em \rightarrow s_1$. Retain all intra-team edges.

We verify $\mathcal{M}_3$ constraints (Definition 8):

- $\mathcal{A}_3$ contains one MANAGER, one INTEGRATOR, one EXPERT MANAGER, and one team with a SUPERVISOR. ✓
- Inter-team edges follow the required pattern. ✓
- $k = |E_1| \geq 1$, satisfying $\mathcal{M}_3$'s structural constraint $k \geq 1$. ✓

Since both new agents have identity policies, $\mathcal{G}_3$ behaves identically to $\mathcal{G}_2$.

Strictness: $\mathcal{M}_2 \neq \mathcal{M}_3$ because multi-team $\mathcal{M}_3$ graphs (with $T > 1$ and non-trivial Integrators) cannot be expressed as $\mathcal{M}_2$ graphs, which lack the Integrator and Expert Manager roles.

Conclusion. $\mathcal{M}_1 \subset \mathcal{M}_2 \subset \mathcal{M}_3$, with each inclusion being strict. □

C.2 Complexity Analysis

Proposition 2 (Communication Complexity). *The worst-case number of inter-agent messages per user request is:*

- $\mathcal{M}_1$: $O(k)$ where $k \leq 2$
- $\mathcal{M}_2$: $O(k)$ where $3 \leq k \leq 5$
- $\mathcal{M}_3$: $O(T + k_{\text{max}})$ where T is the number of teams and $k_{\text{max}} = \max_t |E_t|$

In contrast, a flat fully-connected MAS requires $O(n^2)$ messages where $n = |\mathcal{A}|$.

Proof. In $\mathcal{M}_1$, a request traverses: Manager $\rightarrow$ Expert (1 message per expert, k total) $\rightarrow$ Executors/Perceivers (constant per expert). Total: $O(k)$.

In $\mathcal{M}_2$, a request traverses: Manager $\rightarrow$ Supervisor (1) $\rightarrow$ selected Experts ($\leq k$) $\rightarrow$ Executors/Perceivers. Total: $O(k)$.

In $\mathcal{M}_3$, a request traverses: Manager $\rightarrow$ Integrator (1) $\rightarrow$ Expert Manager (1) $\rightarrow$ relevant Team Supervisor (1) $\rightarrow$ selected team Experts ($\leq k_{\max}$). Cross-team coordination adds $O(T)$ messages. Total: $O(T + k_{\max})$.

In a flat MAS, every agent potentially communicates with every other, yielding $O(n^2)$ in the worst case. Since $T + k_{\max} \ll n^2$ for practical configurations, HYMOEX's hierarchical routing is strictly more efficient. $\square$